我用 14 个 AI Skills员工，建了一条会自我进化的写作流水线

下半部分

一、先统一关于 AI 的概念认知

在讲具体怎么做之前，先把概念搞清楚。

很多人分不清 Workflow、Agent、Skills 这三个东西。我用一句话说清楚：

Workflow 负责提供「约束」和「确定性」。

Agent 负责提供「推理」和「灵活性」。

Skills 负责专业技能。

什么意思？

Workflow 是流水线。它规定了第一步干什么、第二步干什么、出了问题怎么处理。它不会思考，但它保证结果可预测。

Agent 是脑子。它可以根据情况做判断、做推理、选择不同的路径。它很灵活，但有时候不可控。

Skills 是员工。每个员工有自己的专业能力：有的会写文章、有的会审稿、有的会做图、有的会发布。它们被调用来干活。

把这三个搞清楚，你就理解了我这套系统的底层逻辑：

Workflow 是老板的管理制度，Agent 是老板的大脑，Skills 是老板的员工。

二、Skill 的基础知识：四件法宝

Skill 的基础知识网络上铺天盖地，这里仅作简单解释。

Skill 就是一个文件夹。

核心有四样法宝：

第一：SKILL.md

这是 Skill 的说明书。里面用自然语言编写，告诉 AI：这个 skill 是做什么用的、什么时候用、怎么用、要注意什么。

你可以把它理解为新员工的岗位说明书。

第二：scripts（脚本）

就是程序员的常用武器，Python、JS、TS。这些脚本用 AI 都可以口喷出来。

当 AI 要帮你「干活」的时候，就会执行这些脚本。比如调用 API 生成图片、调用 API 发布文章、格式转换等等。

第三：资源文件

你可以放些参考文档、模板配置文件，约束 AI 在查阅这些文档和模板规则下工作。

比如我放了「Forbes 风格指南」，AI 写文章就会参考这个指南。放了「禁止词列表」，AI 就不会用那些词。

第四：渐进式披露

这是一个非常重要的机制。

当你目录下有一百个 skills 的时候，AI 不可能加载一百个 skills。除了 AI 被信息淹没，你的上下文 token 也会爆炸。

它只读取 skill 元数据中的 name、description 描述。

提示词部分只有你需要的时候才会被加载进去，此时提示词才被计算 Token。

如果提示词部分需要加载运行脚本，脚本执行不会被计算 Token。

这就像公司里的员工，老板不需要知道每个员工的详细简历，只需要知道「张三负责财务、李四负责技术」就够了。只有当老板要找张三干活的时候，才需要看张三的详细资料。

三、如何写好 Skill：五大设计原则

要写好 Skill，有五个关键原则：

1. 链式：任务流水的线性拆解。

把一个复杂任务拆成多个步骤，一步一步来。

写文章 → 审稿 → 修改 → 去 AI 味 → 生成配图 → 发布

每一步都是一个独立的 Skill 调用。

2. 路由：像老板一样分发任务。

根据不同情况，调用不同的 Skill。

如果是中文内容，调用中文去味 skill。如果是英文内容，调用英文去味 skill。

这就是老板的核心能力：知人善任，把合适的任务分给合适的人。

3. 并行：不同视角来审查和加速执行。

有些任务可以同时进行。

比如生成四个平台的封面图，不需要一个一个来，可以同时生成。

比如多维度评估，可以同时从品牌一致性、平台匹配度、写作质量等角度打分。

4. 编排：像老板一样能拆解复杂任务给员工。

这是最高级的能力。

一个复杂任务，老板需要知道拆成哪些子任务、每个子任务交给谁、子任务之间有什么依赖关系、最后怎么汇总结果。

比如「发布一篇文章到全平台」这个任务：

- 先生成元文章（生产部）
- 然后评估打分（质控部）
- 然后生成各平台版本（生产部）
- 最后分别发布（发行部）

每个部门有自己的 Skill，老板只需要编排好流程。

5. 评优：自省和自我修正的循环（RLHF Loop）

这是最值钱的能力，后面会详细讲。

以上五点依据 Anthropic 发表的一篇文章《Building effective agents》，感兴趣可以自己搜来看。

四、为什么 99% 的 AI 写作方案都是错的

绝大多数人使用 AI 写作的方式是这样的：

输入 prompt → AI 输出 → 人工修改 → 发布

这个流程有三个致命问题：

问题一：无记忆。

每次写作都从零开始，AI 不知道你上次修改了什么、喜欢什么风格、讨厌什么表达。

你永远在教一个失忆的实习生。

问题二：无标准。

「好」和「不好」全凭感觉。同一篇文章，今天觉得好，明天觉得不好。没有可量化的评估体系，就没有可靠的进化。

问题三：无闭环。

写完就完了。发布后读者反应好不好？哪些规则有效哪些无效？这些数据全部丢失，下次还是凭感觉。

我这套系统的核心设计理念：

把写作从「一次性手工活」变成「可进化的工业流水线」。

五、四层系统架构：像公司一样分工

这套系统由 14 个独立模块组成，但不要把它们当作 14 个独立工具。

正确的理解是：这是一个四层分工的系统。

```
┌──────────────────────────────────────────────────────────────┐
│  第一层：战略层 (Strategy Layer)                                 │
│                                                                 │
│    • 选题分析与趋势研究                                          │
│    • 多学科理论武器库（哲学/心理学/传播学/社会学）                  │
│    • 病毒式钩子生成与 VIRAL 评分                                 │
│                                                                 │
│    核心问题：写什么？用什么角度切入？                              │
├──────────────────────────────────────────────────────────────┤
│  第二层：生产层 (Production Layer)                               │
│                                                                 │
│    • 四维方法论驱动的内容创作                                     │
│    • RLHF 自进化系统（规则持续优化）                              │
│    • 人设管理与品牌一致性                                         │
│                                                                 │
│    核心问题：怎么写？用什么风格？如何保持一致性？                  │
├──────────────────────────────────────────────────────────────┤
│  第三层：质控层 (Quality Control Layer)                          │
│                                                                 │
│    • 7 维度内容评分 + 动态权重调整                                │
│    • Google E-E-A-T 标准审核 + SEO 审计 + 自动重写               │
│    • 24 种 AI 味模式识别与自然化处理                              │
│                                                                 │
│    核心问题：写得好不好？符不符合平台算法？像不像人写的？         │
├──────────────────────────────────────────────────────────────┤
│  第四层：分发层 (Distribution Layer)                             │
│                                                                 │
│    • 微信公众号发布（API 自动化）                                 │
│    • WordPress 发布（REST API + Yoast SEO 元数据）               │
│    • 知乎专栏发布（浏览器自动化）                                 │
│    • X Articles 发布（浏览器自动化）                              │
│    • LinkedIn Articles 发布（浏览器自动化）                       │
│    • Substack Newsletter 发布（API 自动化）                       │
│    • 小红书图片生成（Markdown → 3:4 长图，手动发布）              │
│                                                                 │
│    核心问题：发到哪里？如何一键多平台发布？                        │
└──────────────────────────────────────────────────────────────┘
```

为什么每一层都不可或缺？

没有战略层：写得再好，选题错了就是白忙。90% 的爆款是选题决定的，不是文笔。

没有生产层：每次写作风格飘忽不定，读者无法建立信任。没有「人设」，就没有「粉丝」。

没有质控层：自嗨式写作。你觉得好，算法不推荐，读者不买账。更可怕的是 AI 味太重，读者一眼看穿。

没有分发层：一稿多投效率低下。微信格式、知乎格式、小红书格式各不相同，手动改格式浪费大量时间。

六、RLHF 自进化系统：最值钱的那个员工

这是整个系统的灵魂。

为什么需要 RLHF？

先解释一下什么是 RLHF Loop（也常被称为 Ralph Wiggum Loop）。

RLHF Loop 本质是一个不知疲倦的牛马，它也是 Claude Code 缺失的关键组件之一。

最早名字由来是：《辛普森一家》中最愚蠢的角色之一 Ralph Wiggum。他不断失败，犯下愚蠢的错误，却固执地陷入一个无限循环，直到最终成功。

这种孩子般的坚持正是该机制名称的灵感来源。

传统 AI 写作的「失忆症」

场景还原：

你用 AI 写了一篇文章，发现开头太平淡，于是改成了一个更有冲击力的版本。发布后效果很好。

下次写作，你又用同样的 prompt……

AI 又写了一个平淡的开头。你又改了一遍。

问题在哪？

AI 没有记住：「这个用户喜欢有冲击力的开头」。

你的每次修改，对 AI 来说都是「第一次」。

RLHF 解决的核心问题：让 AI 从你的反馈中学习，越用越懂你。

灵感来源：从 OpenAI 的 RLHF 到内容创作

OpenAI 的 RLHF 是什么？

OpenAI 的 RLHF 是「让模型变聪明」：

- 第一步：让模型生成多个回答
- 第二步：人类标注员对这些回答进行排序（哪个更好）
- 第三步：用这些排序数据训练一个「奖励模型」

- 第四步：用奖励模型指导主模型的优化

我们把这个思路改造成适配内容创作场景：

OpenAI 的 RLHF 需要大量标注员和算力来「微调模型」。我们没有这个资源，但我们可以在 prompt 层面实现类似效果。

原理：不改模型，改规则库。

方法：用规则的得分来指导下次创作时的规则选择。

效果：相当于在「提示词」层面做了 RLHF。

我们不需要真的训练模型，只需要记住哪些写作规则有效、哪些无效，然后动态调整规则的优先级。

设计巧妙之处：用规则库的动态调整，模拟了模型微调的效果，但零算力成本。

五阶段进化循环

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│    ┌──────────┐    ┌──────────┐    ┌──────────┐         │
│    │ Generate │──▶│ Feedback │──▶│  Reward  │         │
│    │  生成    │    │  反馈    │    │  建模    │         │
│    └──────────┘    └──────────┘    └──────────┘         │
│         ▲                               │                │
│         │                               ▼                │
│    ┌──────────┐                    ┌──────────┐         │
│    │  Apply   │◀──────────────────│  Update  │         │
│    │  应用    │                    │  更新    │         │
│    └──────────┘                    └──────────┘         │
│         │                               │                │
│         │          ┌──────────┐         │                │
│         └──────────▶│Consolidate│◀────────┘                │
│                    │  整合    │  ← 每 10 次创作触发       │
│                    └──────────┘                          │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Phase 1: Generate（生成）

基于当前规则集生成内容。

- 输入：主题/素材 + 当前规则集
- 输出：文章 + 元数据（本次应用了哪些规则）
- 关键：记录本次生成应用了哪些规则，便于后续归因

Phase 2: Feedback（反馈收集）

收集用户的显式和隐式反馈。

显式反馈：

- 「这里好」「这里不好」
- 「改成这样」
- 评分（如果用户愿意给）

隐式反馈：

- 用户是否要求修改？（无修改 = 正向信号）
- 修改了几轮？（轮次越少越好）
- 用户最终是否采用？

Phase 3: Reward Modeling（奖励建模）

将反馈转化为规则的「得分」。

```
规则得分 = Σ(应用次数 × 反馈信号) / 应用总次数
```

反馈信号量化：

- +1.0 = 明确表扬（「这句很好」）
- +0.5 = 无修改直接采用
- 0.0 = 有小修改
- -0.5 = 大幅修改
- -1.0 = 明确批评（「这里很差」）

Phase 4: Update（策略更新）

根据得分更新规则集。

得分区间	置信度	操作
> 0.7	> 80%	强化：提升优先级，考虑泛化
0.3~0.7	> 80%	保持：继续观察
< 0.3	> 80%	弱化：降低优先级，考虑淘汰
任意	< 50%	探索：增加应用机会以收集数据

Phase 5: Consolidate（整合优化）

每 10 次创作触发一次规则整合：

整合操作：

1. 去重合并 - 合并语义相似的规则
2. 冲突解决 - 发现矛盾规则，保留高分规则
3. 层级整理 - 将规则组织成树状结构
4. 死规则清理 - 删除长期未使用或持续低分的规则
5. 过拟合检测 - 识别过于具体、不可泛化的规则

规则的数据结构

每条规则不是简单的一句话，而是一个完整的数据对象：

```
- id: rule_010
  category: 金句                      # 规则类别
  content: "金句要够狠，能让读者截图发朋友圈"  # 规则内容
  priority: high                     # 优先级
  score: 0.69                        # 进化得分（0-1）
  confidence: 0.88                   # 置信度（应用次数越多越高）
  applied_count: 7                   # 应用次数
  positive_feedback: 6               # 正向反馈次数
  negative_feedback: 1               # 负向反馈次数
  neutral: 1                         # 中性反馈次数
  created_at: '2026-01-11'           # 创建时间
  last_applied: '2026-01-13'         # 最后应用时间
  parent_rule: null                  # 父规则（泛化来源）
  child_rules: []                    # 子规则（特化版本）
  examples:
    good:                            # 正例
      - "看健身视频不会让你变强壮"
      - "你的答案，就是你的护城河"
    bad: []                          # 反例
```

规则的泛化与特化

泛化：从具体到抽象

当一个具体规则得分高时，尝试提取更抽象的规则：

```
具体规则（得分 0.85）:「用程序员的例子效果好」
    ↓ 泛化
中间规则:「用互联网行业的例子效果好」
    ↓ 再泛化
抽象规则:「用读者熟悉的行业例子效果好」
```

特化：从抽象到具体

当一个抽象规则在某些场景失效时，拆分为条件规则：

```
原规则（得分波动大）：「金句要狠」
    ↓ 特化
职场话题：「职场金句要狠，能激发不满情绪」
情感话题：「情感金句要扎心但不要刻薄，避免读者反感」
```

真实进化案例

以下是系统中真实存在的进化记录：

案例一：开头风格的进化

```
rule_006: 口语化表达

进化轨迹：
  v1 (初始): "开头可以用夸张的反应增加真实感"
    └─ 应用实例: "看到这个消息，我差点咖啡喷出来"
    └─ 用户反馈: "太做作，一看就是编的" → 信号 -0.5

  v2 (修正): "开头要自然，以已经了解很久的视角切入"
    └─ 应用实例: "Linus 用 AI 写代码不是新闻，但这次不一样"
    └─ 用户反馈: "好多了，这个开头自然" → 信号 +1.0

得分变化: 0.4 → 0.65
关键洞察: 「真实感」不等于「夸张反应」，而是「熟悉者的淡定」
```

案例二：人设一致性规则的诞生

```
rule_021: 人设一致性（新规则，从失败中诞生）

触发事件：
  - 文章主题: "2026年用一天时间搞清楚自己想要什么"
  - 问题段落: "我翻出去年的年度目标，发现和前年一模一样"
  - 用户反馈: "这和作者的成功独立开发者人设冲突"

规则创建：
  content: "从「我」视角叙事时，必须保持人设和数据的一致性。
            作者人设是成功的独立开发者（副业年入200万、有盈利产品），
            不是失败者或迷茫者。
            如需描写失败/迷茫状态，用「朋友」或「以前的我」叙事。"

  examples:
    good:
      - "朋友年薪百万，年度目标年年重来。两年前的我也这样，
         后来辞职做独立开发，第一年副业做了200万。"
    bad:
      - "我翻出去年的年度目标，发现和前年一模一样"

得分: 0.75（置信度 50%，需更多数据验证）
```

案例三：文章类型的条件规则

```
rule_020: 干货类文章规则

诞生背景：
  - 原规则: "必须使用众生相技法增加共鸣"
  - 应用场景: 方法论教程文章
  - 用户反馈: "干货类文章众生相不是必须，方法论阐述更重要"

规则特化：
  content: "干货类文章（方法论、教程、工具介绍）：
            方法论和概念阐述优先于众生相，众生相可精简或省略"
  conditions:
    - "文章类型为干货/教程/方法论"

影响：众生相规则不再无条件应用，而是根据文章类型判断
```

冷启动与探索机制

新用户冷启动策略：

```
第 1-5 次创作：
  - 使用默认规则集
  - 强制收集结构化反馈
  - 每次必须回答：「哪里好？哪里不好？」

第 5 次后：
  - 启用完整 RLHF 循环
  - 规则开始有得分和置信度
  - 系统开始「偏好」高分规则
```

探索机制（避免局部最优）：

每 5 次创作进行一次「探索」：

```
探索策略：
  1. 随机选择一个低置信度规则强制应用（收集更多数据）
  2. 或尝试一个规则的变体版本（A/B 测试）
  3. 记录探索结果，更新规则得分

目的：避免系统只用「安全」规则，导致内容同质化
```

进化报告示例

系统可以生成这样的进化报告：

```
## 📈 进化报告

**总创作次数：** 7 次
**平均修改轮次：** 1.3 轮
**采用率：** 87%

### 高分规则 TOP 5
1. [0.75] 禁止使用破折号、冒号、双引号
2. [0.75] 禁止使用首先、其次、此外
3. [0.69] 金句要够狠，能让读者截图发朋友圈
4. [0.67] 嫁接当下热点词汇增加时效性
5. [0.65] 引用权威人物时注明具体态度和限定条件

### 待验证规则（低置信度）
1. [?] 干货类文章众生相可精简（仅应用 1 次，置信度 50%）
2. [?] 人设一致性规则（仅应用 1 次，置信度 50%）

### 进化建议
- rule_006 从 0.4 进化到 0.65：开头风格已校正
- rule_021 从失败中诞生：人设一致性问题已覆盖
- 建议对 rule_020 增加应用机会以验证效果
```

用了 7 次之后，我的系统已经知道我喜欢什么风格的开头、什么样的金句、什么结构最有效。

它不再是失忆的实习生，而是一个越用越懂我的写作搭档。

七、战略层：选题的科学化

不是找热点，而是「理论武装」

市面上的选题工具做的是什么？爬热点、抓热搜、看竞品。

我的选题逻辑完全不同：

用学术理论武装选题，让每个选题都有「深度锚点」。

四维理论武器库

维度	不用理论的选题	用理论武装的选题
哲学	「年轻人不想上班了」	「年轻人不想上班了，海德格尔早就预言过」——向死而生
心理学	「为什么你总是拖延」	「拖延不是懒，是你的阴影在保护你」——荣格阴影理论
传播学	「如何写出爆款」	「爆款的本质是认知失调」——框架效应
社会学	「中产焦虑」	「中产焦虑的本质是文化资本焦虑」——布尔迪厄

核心洞察：理论不是为了装逼，是为了提供深度锚点。

有锚点的文章，读者看完有「恍然大悟」的感觉；没锚点的文章，看完就忘。

VIRAL 评分矩阵

选题不是拍脑袋，而是可量化评估的。

```
V (Validation/Violation) - 期望验证/违反程度  × 1.5
I (Identity)             - 身份激活程度      × 1.2
R (Resonance)            - 情感共鸣程度      × 1.3
A (Attention)            - 注意力捕获力      × 1.5
L (Logic)                - 逻辑可信度        × 0.5

VIRAL Score = (V×1.5 + I×1.2 + R×1.3 + A×1.5 + L×0.5) / 6

目标：≥ 7.5 具有高爆款潜力
```

为什么这个公式有效？

- V 和 A 权重最高（1.5）：因为在注意力经济时代，「能被看到」比「内容好」更重要
- L 权重最低（0.5）：因为情绪驱动分享，不是逻辑驱动分享
- I 和 R 中等权重：身份认同和情感共鸣是分享的核心动机

八、质控层：三道关卡让 AI 味无处藏身

初稿写完，进入质控。

我设计了三道关卡。

第一关：7 维度评分

维度	权重	为什么需要这个维度
品牌一致性	10%	没有一致性就没有粉丝
平台匹配度	15%	算法不推，内容再好也没用
写作质量	15%	基础功，逻辑/深度/技巧
互动潜力	18%	权重最高——互动决定推荐
风险评估	15%	一次翻车毁所有
事实核查	12%	尤其 YMYL 领域
时效性	15%	热点有保质期

每个维度打分，综合评分低于 9 分就打回重写。最多三轮。

第二关：AI 去味

读者对 AI 内容的容忍度在急剧下降。

我整理了 24 种 AI 味模式：

模式	典型表现	为什么令人反感
夸大象征	「标志着」「见证了」「关键时刻」	明明是小事，说得像历史转折
三段式法则	「创新、灵感、行业洞察」	机械感太强，一看就是 AI
否定式排比	「这不仅仅是X，而是Y」	故弄玄虚，内容空洞
-ing 肤浅分析	「象征着...」「反映了...」	假装有深度，实际是废话
模糊归因	「专家认为」「研究表明」	没有具体来源，可信度为零

逐条检测，逐条替换。

第三关：SEO 和 E-E-A-T 审核

Google 的 E-E-A-T 标准：

维度	核心问题
YMYL 影响	这内容会影响读者的钱和健康吗？
内容质量	有深度吗？有原创见解吗？
E-E-A-T	有专家资质吗？引用权威来源吗？
用户参与	有互动元素吗？有 CTA 吗？
原创价值	提供独特见解还是换皮搬运？
结构可读	标题层级合理吗？好浏览吗？
透明信任	有免责声明吗？广告披露了吗？
元数据 SEO	Title/Meta 优化了吗？

自动重写触发：内容质量/原创价值/元数据 SEO 任一项 < 8 分 → 系统自动重写全文。

三关过完，才能进入发布。

九、分发层：7 个平台的自动化

自动化程度矩阵

```
                完全自动化                      半自动化                      手动
                   |                              |                          |
┌──────────────────┼──────────────────────────────┼──────────────────────────┐
│                  |                              |                          |
│  WordPress       |  X Articles                  |  小红书                  |
│  微信公众号      |  LinkedIn                    |                          |
│  Substack        |  知乎                        |                          |
│                  |                              |                          |
│  (API 调用       |  (浏览器自动化               |  (生成图片               |
│   无需浏览器)    |   需要 Chrome)               |   返回目录)              |
│                  |                              |                          |
└──────────────────┼──────────────────────────────┼──────────────────────────┘
```

为什么需要 7 个独立的发布模块？

每个平台的技术栈、格式要求、自动化可行性都完全不同：

平台	技术方案	为什么选择这个方案
微信公众号	REST API	有官方 API，最稳定
WordPress	REST API + Yoast	行业标准，支持 SEO 元数据
知乎	Chrome CDP	无公开 API，只能浏览器自动化
X Articles	浏览器自动化	API 限制太多，自动化更灵活
LinkedIn	浏览器自动化	Articles API 不开放
Substack	API	支持 Newsletter 草稿创建
小红书	图片生成	无 API，生成长图后手动发布

十、三个设计理念

理念一：流水线思维

不是「用 AI 写文章」，而是「建立一条会自我进化的生产线」。

```
选题 → 写作 → 审稿 → 去味 → SEO优化 → 多平台发布
  │       │       │       │         │          │
  └───────┴───────┴───────┴─────────┴──────────┘
                          │
                     反馈闭环
                          │
                 规则进化 + 人设积累
```

理念二：机器做重活，人做决策

机器负责	人负责
生成初稿	确认战略方向
评估打分	审核最终质量
格式转换	选择发布平台
规则进化	重大规则变更

导演不用自己演戏，但导演决定这场戏怎么演。

理念三：数据资产思维

每次使用，都应该产生可积累的资产：

- 进化的规则（越用越准确）
- 积累的人设（越写越有辨识度）
- 验证的模式（越多越知道什么有效）

十一、与普通 AI 写作的本质区别

维度	普通 AI 写作	这套系统
记忆	无	RLHF 进化系统
标准	主观	7 维度评分 + E-E-A-T
风格	飘忽	人设系统锁定
深度	随机	理论武器库保底
AI 味	明显	24 模式去味
发布	手动	7 平台自动化

一句话总结

这不是让 AI「帮你写」，而是让 AI「成为你的写作分身」——有你的风格、你的标准、你的积累，而且会不断进化。

如果你也想建一套这样的系统，你最想让 AI 员工干什么活？

评论区告诉我。